

MTGNP Project Structure

```
mtgnp/
├── __init__.py          # Package exports
├── __main__.py          # Entry point for python -m mtgnp
├── main.py              # CLI entry point
├── launcher.py          # Script launcher
├── README.md
├── core/                # Core models and utilities
│   ├── __init__.py
│   ├── constants.py     # Constants and configuration
│   ├── game_logger.py   # Game logging/replay functionality
│   ├── models.py        # Data models (Permanent, StackItem, Player)
│   └── utils.py          # Utility functions
├── network/             # Networking components
│   ├── __init__.py
│   ├── client.py        # Client implementation
│   ├── network.py       # Network utilities (encoding/decoding)
│   ├── server.py        # Server implementation
│   └── spectator.py     # Spectator client
└── game/                # Game engine logic
    ├── __init__.py      # Game module exports
    ├── actions.py       # Player action handlers
    ├── card_catalog.py  # Complete card definitions (100+ cards)
    ├── card_effects.py  # Card effect implementations (25+ effects)
    ├── combat.py        # Combat system
    ├── game.py          # Main game orchestrator
    ├── lifecycle.py     # LOBBY, SETUP, MULLIGAN, GAME_OVER
    ├── priority.py      # Priority and stack management
    ├── state.py         # Game state management
    ├── triggers.py      # Triggered abilities manager
    └── turn.py          # Turn and phase engine
```

File Descriptions

Root Level Files

File	Description
<code>__init__.py</code>	Package root, exports all public classes
<code>__main__.py</code>	Entry point for <code>python -m mtgnp</code>
<code>main.py</code>	CLI entry point with argument parsing
<code>launcher.py</code>	Script launcher

Core Module (`core/`)

File	Description
<code>constants.py</code>	All constants (ports, timeouts, phases, error codes)
<code>game_logger.py</code>	Game logging, saving, loading, replay functionality
<code>models.py</code>	Data models: <code>Permanent</code> , <code>StackItem</code> , <code>Player</code>
<code>utils.py</code>	Utility functions (shuffle, draw, mana checking, etc.)

Network Module (`network/`)

File	Description
<code>client.py</code>	MTGNP client - connects to server, sends actions, renders state
<code>network.py</code>	Network utilities: <code>encode_message</code> , <code>decode_message</code> , <code>send_pdu</code>
<code>server.py</code>	MTGNP server - handles connections and routes PDUs
<code>spectator.py</code>	Read-only spectator client with history/replay

Game Module (`game/`)

File	Description
<code>actions.py</code>	Player action handlers (cast spell, play land, declare attackers, discard)
<code>card_catalog.py</code>	Complete card catalog with 100+ cards, card lookup functions
<code>card_effects.py</code>	All card effect implementations (25+ effects)
<code>combat.py</code>	Combat system (damage calculation, resolution)
<code>game.py</code>	Main game orchestrator - coordinates all sub-systems
<code>lifecycle.py</code>	Game lifecycle (LOBBY, SETUP, MULLIGAN, GAME_OVER)
<code>priority.py</code>	Priority, stack management, SBA loop, and fizzle logic
<code>state.py</code>	Game state management (players, permanents, state building)
<code>triggers.py</code>	Triggered ability manager (ETB, attack, death triggers)
<code>turn.py</code>	Turn and phase engine (all phases and steps)

Running the Project

Prerequisites

- Python 3.x
- No external dependencies (uses only Python standard library)

Start Server

```
python launcher.py server --port 4444 --verbose
python launcher.py server --port 4444 --verbose --log-file game.log
```

Start Client (Player 1)

```
python launcher.py client --host localhost --port 4444 --player player_1 --deck
lightning_bolt_001 lightning_bolt_002 shock_001 mountain_001 mountain_002
goblin_guide_001 --verbose
```

Start Client (Player 2)

```
python launcher.py client --host localhost --port 4444 --player player_2 --deck
counterspell_001 island_001 swamp_001 gray_merchant_001 mountain_003 shock_002 --
verbose
```

Start Spectator

```
python launcher.py spectator --host localhost --port 4444 --verbose
```

List Available Cards

```
python launcher.py --list-cards
```

Deck Requirements

- Minimum deck size: 1 card
- Maximum deck size: 50 cards
- Cards must be valid card IDs from the catalog
- See full card list with `--list-cards`

Client Commands

Command	Description
<code>help</code>	Show available commands
<code>hand</code>	Show your hand
<code>state</code>	Show full game state
<code>pass</code>	Pass priority

Command	Description
concede	Concede the game
cast <card_id> [targets...]	Cast a spell
land <card_id>	Play a land
attack <creature_id> [target]	Declare attacker
block <creature_id> <attacker_id>	Declare blocker
discard <id1> [id2...]	Discard cards (used during Cleanup step)
list	List available cards
quit/exit	Exit client

Spectator Commands

Command	Description
state	Show game state
history	Show history count
save <file>	Save history to file
load <file>	Load history from file
replay	Replay game
quit/exit	Exit spectator

Project Statistics

- **Total Files:** 22
- **Card Catalog:** 100+ unique cards
- **Card Effects:** 25+ implemented
- **Game Phases:** 14 phases/steps
- **Error Codes:** 11 defined
- **PDU Types:** 26 defined

Work Distribution Matrix

- Member 1: CRISOLOGO, PAUL MARTIN RYAN
- Member 2: MARQUEZ, JAMES BENEDICT
- Member 3: OCAMPO, JERICHO
- Member 4: RAYMUNDO, MARTINA GABRIELLE

Task / Feature	Member 1	Member 2	Member 3	Member 4
TCP Server: connection handling, framing, dispatch	X	X		

Task / Feature	Member 1	Member 2	Member 3	Member 4
Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic		X		
Turn & phase engine (all phases/steps, transitions)		X	X	
Priority & Stack logic, spell/ability resolution	X		X	
Combat system (attackers, blockers, damage)			X	
Client implementation & state rendering	X			
PDU serialisation/deserialisation (all 25 PDU types)				X
Error handling, PING/PONG heartbeat, disconnect logic				X
Verbose mode (client + server PDU logging, toggle on/off)	X			
Testing & interoperability			X	X
README / documentation / AI disclosure		X		
Bonus Features: Advanced mechanics, custom card effects, and additional combat steps				X

AI Usage

- **Gemini:** Used it for understanding the specs and rechecking if everything is implemented. Also used it to format the README.md and for debugging.
- **Codex:** Used for debugging and code creation for complex logic.
- **Claude:** Used it for understanding the specs and rechecking if everything is implemented. Also used for debugging and code creation for complex logic.

Known Limitations / Deviations from RFC

While the MTGNP 1.0 protocol is strictly compliant in its foundational engine (priority passes, network framing, sequence numbering), several **intentional deviations** were introduced to support complex bonus card effects not covered by the original specification:

- **First Strike / Assign Damage Order:** To support cards with First Strike (e.g., *White Knight*, *Black Knight*), two additional combat steps (*ASSIGN_DAMAGE_ORDER* and *FIRST_STRIKE_DAMAGE*) were added to the phase progression. The base RFC defines only 14 steps, whereas this implementation has 16 to properly resolve First Strike mechanics.
- **Colored Mana Tracking:** The RFC implies generic mana tracking, but the engine implements fully distinct colored mana pools (W, U, B, R, G) to handle specific casting costs and devotion.
- **Advanced State Mechanics:** The protocol state was augmented with *_cannot_gain_life* (for *Skullcrack*), *_prevent_next_damage* (for *Healing Salve*), and *get_devotion()* calculations (for *Gray Merchant of Asphodel*). These fields extend the state beyond the base RFC's *Permanent* definition but are required for the bonus features to function correctly.
- **Flying / Evasion:** Any creature can currently block a flyer.

- **Vigilance:** Attacking still taps the creature.
- **Hexproof / Protection:** The `_protected` flag is set by certain card effects but never verified in target validation.
- **Kicker / Madness / Suspend:** Tagged on some cards but there are no PDU fields or code paths to invoke them.
- **Trample:** Intentionally absent, as the RFC (Section 1) explicitly excludes it from MTGNP 1.0.